Casco Magnificado

Una interfaz corporeizada para Pierre Flutist

Patricio de la Cuadra — con apoyo de Claude Code

Versión 0.1 — Abril 2026

Resumen en una línea. Reemplazar el control por PS Move por un arnés cefálico ligero que instrumenta la cabeza del flautista: el soplo se captura con un sensor solidario a la boca, y la geometría 3D del embocador se reconstruye magnificada a partir de la pose de la cabeza respecto a un bisel virtual fijo frente al músico.

Índice

1. Resumen ejecutivo **3**

2. Motivación **3**
2.1 El problema con PS Move . . . 3
2.2 Lo que se aprendió del intento de cabezal sensorizado . . . 3
2.3 Lo que motiva el casco magnificado . . . 4

3. Concepto: la flauta magnificada **4**
3.1 Metáfora . . . 4
3.2 Diagrama conceptual . . . 4
3.3 Grados de libertad del usuario -> grados de libertad del robot . . . 5

4. Diseño del arnés cefálico **5**
4.1 Principios de diseño . . . 5
4.2 Arquitectura mecánica . . . 5
4.3 Distribución de peso . . . 6
4.4 Diseño del boom y el sensor de soplo . . . 7

5. Arquitectura del sistema **7**
5.1 Visión por capas . . . 7
5.2 Dónde encaja en el código existente . . . 8
5.3 Modelo temporal . . . 8
5.4 Recuperación de fallas . . . 8

6. Hardware (BOM) **8**
6.1 Componentes principales . . . 9
6.2 Pesos y balance . . . 9

7. Firmware del ESP32 **10**

8. Integración con la arquitectura de Pierre **10**
8.1 Nuevo archivo: `src/helmet.py` . . . 10
8.2 Función de mapeo magnificado . . . 12
8.3 Cambios en `main.py` . . . 12
8.4 Cambios en `src/musician.py` . . . 13
8.5 Cambios en la GUI . . . 13

9. Calibración y mapeo de gestos **13**
9.1 Calibración de instalación (una sola vez) . . . 13
9.2 Calibración por sesión (cada vez que se enciende) . . . 14
9.3 Construcción de diccionario de notas (caso de uso clave) . . . 14

10. Evaluación honesta de capacidades **15**
10.1 Zona VERDE — medibles con alta confianza . . . 15
10.2 Zona AMARILLA — depende de detalles de implementación . . . 15
10.3 Zona ROJA — NO medibles con esta propuesta . . . 16
10.4 Conclusión honesta . . . 16

11. Plan de validación fail-fast **16**
Test H.1 — ¿El IMU en la cabeza es suficientemente estable? (2 días) . . . 16
Test H.2 — ¿El pitot montado en boom detecta vibrato? (3 días) . . . 17
Test H.3 — ¿ArUco múltiple sobrevive a rotaciones de cabeza? (2 días) . . . 17
Test H.4 — Prueba conceptual offline (3 días) — EL MÁS IMPORTANTE . . . 17
Test H.5 — Latencia end-to-end (1 día) . . . 17
Test H.6 — Aceptación y confort (2 días) . . . 17

12. Fases de implementación **18**

13. Estimaciones de éxito y riesgos **18**
13.1 Probabilidades subjetivas . . . 18
13.2 Riesgos técnicos principales . . . 19
13.3 Riesgos de proyecto . . . 19

14. Comparación con alternativas evaluadas **19**

15. Decisiones abiertas **20**
D.1 — ¿Cableado USB o wireless? . . . 20
D.2 — ¿Cámara en el atril o en el techo? . . . 20
D.3 — ¿Mantener el modo PS Move o reemplazarlo? . . . 20
D.4 — ¿Quién ejecuta cada fase? . . . 21
D.5 — ¿Publicación? . . . 21
D.6 — Presupuesto . . . 21
D.7 — ¿Casco abierto (arnés) o cerrado (casco completo)? . . . 21

Próximos pasos sugeridos **21**

1. Resumen ejecutivo

Pierre Flutist es un robot flautista de 9 grados de libertad que hoy puede ejecutar partituras pregrabadas y operarse en modo PS Move, donde un control de PlayStation manipulado por el usuario se traduce en movimientos del robot. El modo PS Move se usa principalmente para construir diccionarios de notas (ajuste manual hasta que cada nota suena afinada y timbrada) y para demos públicas.

Una propuesta previa (*Cabezal Sensorizado*, v0.1) planteaba reemplazar el PS Move por una cabeza de flauta instrumentada que el usuario sostiene como una flauta real. Tras revisión técnica, esa propuesta se descarta por una razón arquitectónica: el mismo objeto tenía que cumplir dos funciones incompatibles — (a) capturar el chorro de aire del flautista, y (b) representar la geometría 3D del embocador. Un pitot montado en un objeto con geometría móvil y orientación impredecible tiene un pronóstico pobre de captura limpia del soplo.

Esta nueva propuesta, **Casco Magnificado**, separa las dos funciones en sensores independientes:

- El **soplo** se captura con un sensor de presión diferencial montado solidariamente a la boca del usuario mediante un boom ligero, en geometría fija — el sensor siempre ve un chorro de aire estable y en la misma posición relativa a los labios.
- La **geometría 3D** del embocador se reconstruye a partir de la pose (posición y orientación) de la cabeza del flautista respecto a un punto virtual fijo — *el bisel magnificado* — ubicado a ~20 cm frente al músico, sobre su atril.

Tesis central. Si un flautista puede ”apuntar” con su cabeza hacia un bisel imaginario agrandado 10 veces, sus gestos naturales de afinación y dinámica se traducen 1-a-1 a los grados de libertad del robot — sin traducciones mentales, sin mapeos arbitrarios.

Este documento detalla la arquitectura hardware/software, el diseño específico del arnés cefálico (ligero, ajustable, no invasivo), el protocolo de comunicación, la matemática del mapeo magnificado, un plan de validación fail-fast y una evaluación honesta de qué aspectos del arte flautístico sí podremos capturar y cuáles probablemente no.

2. Motivación

2.1 El problema con PS Move

El modo PS Move actual depende de hardware discontinuado (PS Move de 2010, cámara PS Eye), un subproceso binario externo (`psmove.exe`) que se comunica por UDP, y tracking HSV por color — sensible a iluminación. El mapeo es contra-intuitivo: mover una esfera de plástico en el espacio 3D no se parece en nada a tocar la flauta.

2.2 Lo que se aprendió del intento de cabezal sensorizado

La propuesta previa dio dos lecciones importantes:

1. **Separación de concerns** — un sensor de presión para capturar el chorro del flautista debe estar en geometría fija respecto a la boca. Montarlo en un objeto móvil y rotable introduce ambigüedad aerodinámica irresoluble.

2. **Manos libres importan** — sostener un cabezal de flauta en la mano y soplarle requiere una postura contorsionada que no corresponde a cómo el flautista toca realmente. Las manos deberían quedar libres para (en el futuro) manipular una interfaz de dedos.

2.3 Lo que motiva el casco magnificado

Un flautista profesional ya tiene entrenada, en su cuerpo, toda una coreografía:

- Gira levemente la cabeza para corregir la afinación (cambia el ángulo del chorro sobre el bisel).
- Inclina la barbilla hacia arriba o hacia abajo para octavar o cambiar el timbre.
- Acerca o aleja la cabeza de la flauta para modular la dinámica.
- Modula el soplo con el diafragma para vibrato.
- Articula con la lengua para ataques.

Estas acciones no se hacen con la mano — se hacen con la cabeza y el aire. El casco magnificado captura exactamente las mismas acciones, en la misma parte del cuerpo, pero las mapea a un embocador virtual escalado ×10, permitiendo que gestos cómodos produzcan control fino.

3. Concepto: la flauta magnificada

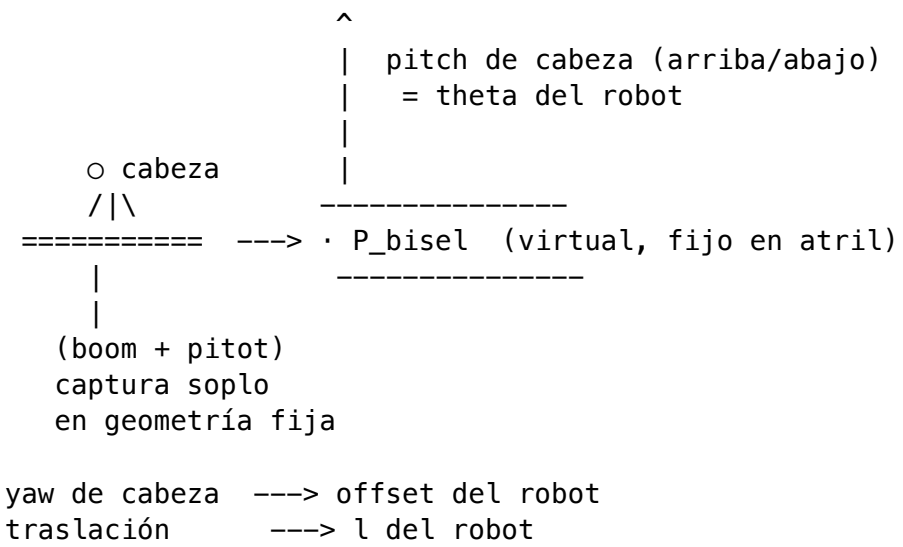
3.1 Metáfora

Imagine el lector que su embocadura tiene 10 cm de largo (en vez de 1 cm), el bisel es una cuchilla gigante sostenida en un atril a 20 cm frente a su boca, y su chorro de aire se extiende como un haz visible desde los labios. Tocar consistiría en:

- Orientar la cabeza de modo que el chorro apunte al bisel virtual.
- Inclinar la cabeza arriba/abajo para pasar el chorro por encima o por debajo del filo.
- Acercar o alejar la cara para variar la distancia labios-bisel.
- Soplar más fuerte o más suave, igual que siempre.

Un flautista entiende esta metáfora sin explicación. Un ingeniero que no sabe tocar flauta, no.

3.2 Diagrama conceptual



```
presión boom    ---> flujo + vibrato
```

3.3 Grados de libertad del usuario -> grados de libertad del robot

Acción del flautista	Sensor	Variable robot
Girar cabeza (yaw)	IMU — quaternion	Offset lateral + componente de α
Inclinar cabeza (pitch)	IMU — quaternion	Ángulo θ de incidencia
Rolar cabeza (roll)	IMU — quaternion	(reservado, opcional: balance)
Acercar/alejar cara	Pose visual (ArUco)	Distancia l labios-bisel
Trasladar lateralmente	Pose visual (ArUco)	Offset adicional
Soplar (DC)	Boom + MPX5010DP	Flujo principal (SCCM)
Modular flujo (AC)	Boom + FFT	Vibrato: freq + amplitud
Articular con lengua	Boom — transitorios rápidos	Mordiente (*yellow*, ver §10)
Botón capacitivo	Touch en sien	Graba punto en diccionario

Principio de diseño. Cada gesto natural del flautista mapea al grado de libertad del robot que afectaría acústicamente de forma análoga. Nada de botones para ”activar vibrato”: si el usuario modula su aliento, el robot modula su flujo.

4. Diseño del arnés cefálico

Este es el componente que más debe pensarse desde ergonomía. Un casco torpe arruina la propuesta entera, por buena que sea la ingeniería detrás.

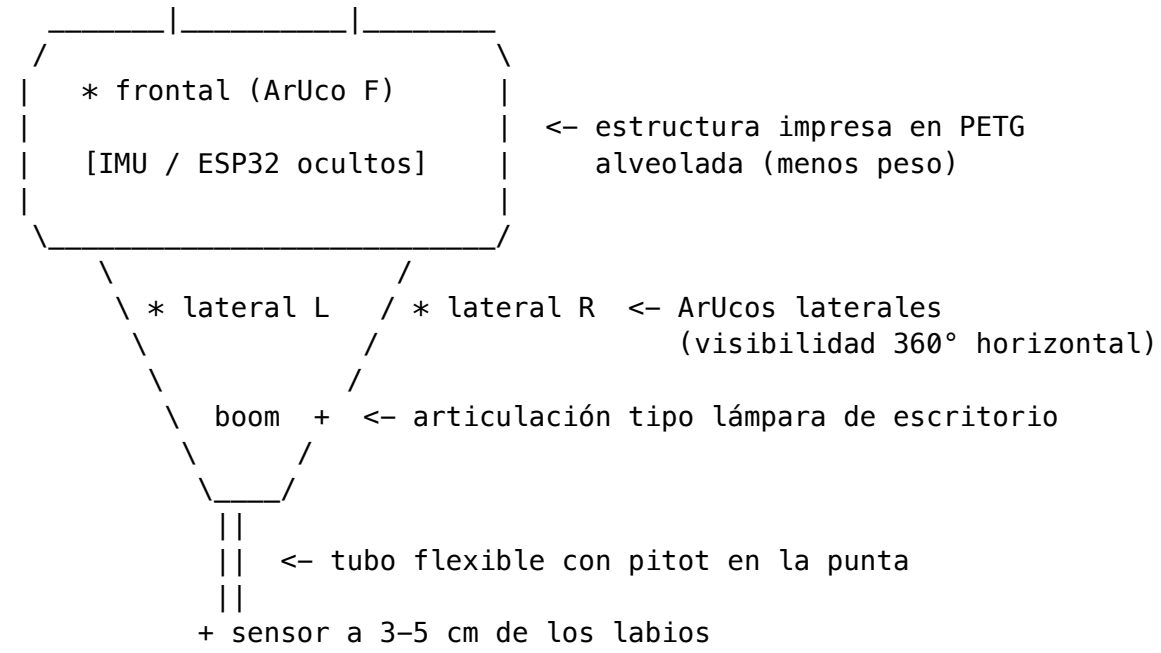
4.1 Principios de diseño

1. **Ligero**: objetivo < 250 g en la cabeza completa (con boom, electrónica y cable). Punto de comparación: unas gafas VR tipo Oculus Go pesaban 468 g y eran incómodas tras 30 min; unos cascos de audio pro son ~300 g.
2. **Ajustable**: una sola talla debe servir a cabezas entre 55 y 60 cm de circunferencia (población chilena adulta, percentil 5-95).
3. **No invasivo**: no debe apretar la cabeza, ni tocar las orejas, ni taparlas. El flautista tiene que poder escuchar al robot con fidelidad.
4. **Reversible**: debe poderse quitar en <5 segundos (botón rápido tipo snap).
5. **Estéticamente aceptable para demos públicas**: no debe parecer un experimento de laboratorio. Un look de “auriculares profesionales con un micrófono boom” es ideal.

4.2 Arquitectura mecánica

Recomiendo NO usar un casco cerrado tipo football americano (pesado, caliente, acústicamente opresivo). La referencia ergonómica adecuada es un **arnés de estudio de cine / VR DK tipo Valve Index o un Head-Strap de ski goggles**:

```
Banda frontal (ajustable con dial occipital)
      +----------+
```



Componentes:

Componente	Material	Peso est.
Banda frontal ajustable (dial occipital)	Plástico ABS + almohadilla espuma	80 g
Estructura rígida superior (donde van sensores)	PETG impreso 3D (paredes 2 mm, patrón giroid)	60 g
IMU BNO085 + ESP32-S3 + PCB	Perfboard o PCB custom	25 g
Boom articulado (3 juntas ajustables)	Cuello de ganso metálico o varillas impresas	40 g
Sensor MPX5010DP + housing pitot	PLA + conector tubo silicona	15 g
Cableado USB-C + cable interno	Cable flexible apantallado 1.5 m	30 g
TOTAL estimado		**~250 g**

4.3 Distribución de peso

El centro de masa del arnés debe caer sobre el eje vertical de la cabeza, idealmente ligeramente detrás (occipital), para que el cuello no tenga que compensar una palanca frontal. Esto sugiere:

- ESP32 + batería opcional en el **occipucio** (contrapeso natural del boom).
- IMU en el **vértex** (top de la cabeza) — zona de rotación mínima para un cabeceo, reduce el ruido de traslación al girar la cabeza.
- Boom anclado en la **sien derecha** (convención de micrófonos de headset; deja el lado izquierdo libre para el flautista que toca con su flauta — si quisiéramos extender a tocar con flauta real y robot simultáneo).

4.4 Diseño del boom y el sensor de soplo

El boom es el único componente que toca aire soplado, y tiene que cumplir una condición no trivial: **medir el chorro sin imponer resistencia que altere la sensación del flautista**.

Dos opciones de arquitectura para el captador:

A. Pitot lateral (recomendado para v1):

Un tubo de silicona de 3 mm de diámetro, orientado *paralelo* al chorro esperado del usuario, con su apertura enfrentada al flujo a ~3-5 cm de los labios. Conectado al puerto positivo del MPX5010DP. El puerto negativo queda abierto al ambiente a ~20 cm del boom. Así se mide la presión dinámica (velocidad) del jet sin obstruirlo. El usuario sopla como si no hubiera sensor.

B. Malla de captura (opcional iteración v2):

Un pequeño embudo de silicona muy blanda (diámetro 4 cm en la entrada, 2 mm en la salida) canaliza parte del chorro hacia el sensor. Mayor SNR, pero puede alterar la sensación.

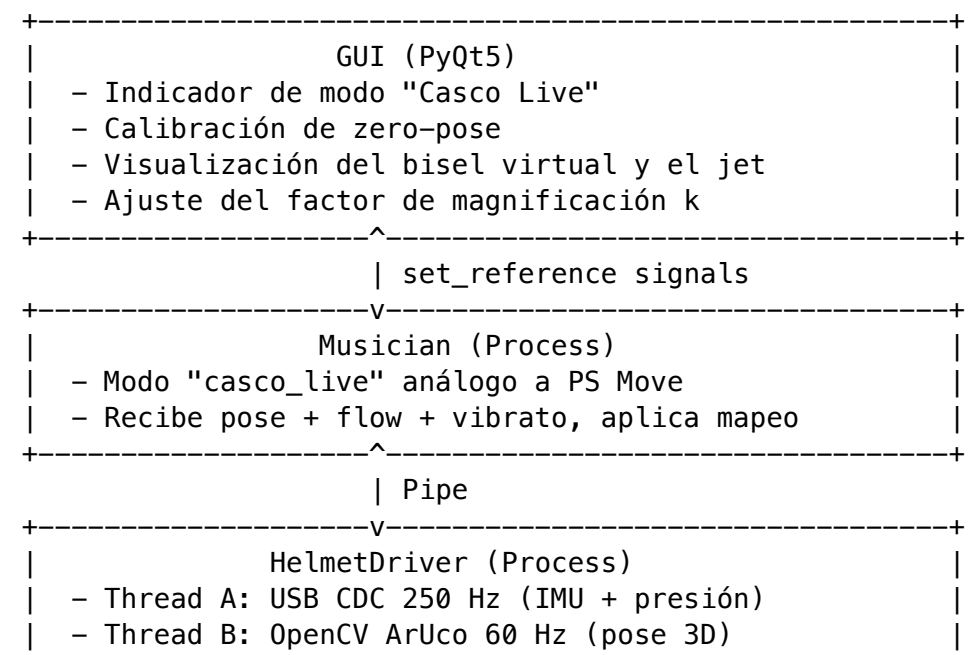
Recomendación firme: empezar con A. Si en H.2 el vibrato no se detecta con SNR > 10 dB, migrar a B o a un sensor mejor (SDP810).

Trampa de humedad: el aliento condensa agua tras 5-10 min de soplo continuo. El diseño debe incluir:

- Tubo en forma de U con punto bajo en la mitad del recorrido (el agua se acumula ahí, no llega al sensor).
- Tornillo de drenaje accesible sin herramientas (inspección cada 20 min).
- Material hidrofóbico en la superficie interior (recubrimiento cerámico-like, o simplemente PTFE líquido).

5. Arquitectura del sistema

5.1 Visión por capas



```
|  - Thread C: Fusión IMU+visual, filtrado,             |
|              mapeo magnificado, envío al musician     |
+--------------^-----------------------^---------------+
               |                       |
[USB CDC serial]                   [Cámara USB en atril]
[ESP32-S3 en casco]                [3 ArUcos en casco]
  |-- BNO085 IMU (I2C)
  |-- MPX5010DP presión (ADC)
  |-- Touch capacitivos sien
```

5.2 Dónde encaja en el código existente

La arquitectura de Pierre ya es multiproceso: el `Musician` coordina drivers individuales para motores AMCI, flujo Alicat, y dedos, todos comunicados por `Pipe`. La integración del casco es aditiva — se crea un nuevo driver `HelmetDriver` que vive al mismo nivel que los demás y publica referencias al `Musician` exactamente como hoy lo hace el proceso PS Move. No se reescribe nada del core.

5.3 Modelo temporal

Componente	Frecuencia	Latencia típica
IMU + presión (ESP32 -> PC por USB CDC)	250 Hz	1-3 ms (USB)
ArUco (OpenCV sobre frames USB)	60 Hz	~16 ms (cámara)
Fusión + envío a musician	200 Hz	< 1 ms
PID del robot (drivers.py)	100 Hz	Depende del motor
Latencia total esperada: gesto -> movimiento robot		**15-30 ms**

5.4 Recuperación de fallas

Falla	Detección	Respuesta
USB del casco desconectado	Timeout 100 ms sin trama	Freeze última ref; LED rojo GUI; soft stop tras 1 s
Cámara perdida	OpenCV sin frame > 200 ms	Continuar con IMU solo (orientación); warning GUI
Todos los ArUcos ocultos	Detección vacía > 500 ms	Extrapolación con IMU (dead reckoning limitado); warning
Sensor presión saturado	ADC en rail	Limitar flujo comando; warning sonoro
CRC inválido	Chequeo en driver	Descartar trama; contador de errores

6. Hardware (BOM)

6.1 Componentes principales

Componente	Parte	Precio (USD)	Notas
Microcontrolador	ESP32-S3-DevKitC-1	12	USB CDC nativo, I2C, ADC, BLE opcional
IMU 9-DOF	Adafruit BNO085 breakout	25	Fusión interna 400 Hz, quaternion estable
Sensor presión	MPX5010DP (NXP)	18	Diferencial, 0-10 kPa, ideal para aliento
Pitot / tubo aire	Silicona 3 mm + conector impreso	5	Con trampa de agua
Arnés frontal-occipital	Impreso PETG + dial de talla	12	Diseño custom
Almohadilla frontal	Espuma con microfibra	4	Lavable
Boom articulado	Cuello de ganso 20 cm	8	Tipo micrófono headset pro
Marcadores ArUco	Impresos PLA rígido, 40×40 mm, 3 unidades	3	4×4_50 dictionary
Cámara USB	Logitech C920 (ya en lab) o Arducam	0-80	USB UVC, 60 fps 720p
Cable USB-C	2 m apantallado	8	Bajo ruido EMI
PCB prototipo	Perfboard o PCB JLCPCB 5 unidades	15	Montaje definitivo
Botones capacitivos	Módulos TTP223 ×2	3	Para sienes
Tornillería, inserts M2	Surtido	5	Ensamblaje

Total estimado: US$ 120 - 200 según si se usa cámara nueva y se encarga PCB custom.

6.2 Pesos y balance

Subsistema	Peso	Ubicación
Arnés + almohadilla	~80 g	Circunferencial
Estructura superior + IMU	~60 g	Vértex
ESP32-S3 + PCB	~25 g	Occipital (contrapeso)
Boom + sensor	~55 g	Sien derecha
Cableado interno	~30 g	Distribuido
Total en la cabeza	**~250 g**	Centro de masa ~vertical

Comparación: unos auriculares Sony WH-1000XM5 pesan 250 g y se usan rutinariamente 2+ horas sin fatiga.

7. Firmware del ESP32

Arquitectura idéntica a la propuesta anterior: FreeRTOS con dos tareas principales — lectura de sensores a 250 Hz y envío por USB CDC. Framing binario con magic (0xA5A5), versión, sequence number, timestamp del ESP32 (`micros()`), quaternion fusionado, acelerómetro, giroscopio, presión cruda (ADC 12-bit), botones táctiles (bitfield), y CRC-16-CCITT. Tamaño de trama: 53 bytes. Bandwidth: 13.25 kB/s (trivial para USB CDC a 12 Mbps).

El detalle completo del esqueleto de código Arduino y del formato de trama se conserva sin cambios respecto a la propuesta anterior (§6-7 de *Cabezal Sensorizado v0.1*), porque el microcontrolador no sabe ni le importa si está en un cabezal de flauta o en un casco. La única adición menor es un segundo canal de touch capacitivo (dos botones en vez de uno, para sienes izquierda y derecha).

8. Integración con la arquitectura de Pierre

8.1 Nuevo archivo: `src/helmet.py`

```python
import serial, struct, time, logging
from multiprocessing import Process, Pipe
from collections import deque
import numpy as np
import cv2
from scipy.spatial.transform import Rotation as R

logger = logging.getLogger(__name__)

PACKET_FMT = "<HBBI4f3f3fHBH"   # 53 bytes
PACKET_SIZE = struct.calcsize(PACKET_FMT)
MAGIC = 0xA5A5


class HelmetDriver(Process):
    def __init__(self, port, musician_pipe, cam_index=0, bisel_offset_m=0.20,
                 magnification_k=10.0):
        super().__init__(daemon=True)
        self.port = port
        self.musician_pipe = musician_pipe
        self.cam_index = cam_index
        self.bisel_offset_m = bisel_offset_m  # distancia virtual al bisel
        self.k = magnification_k              # factor de magnificación
        self.running = True

        # calibración (se setea en calibrate_zero())
        self.q_zero = None         # quaternion de pose neutral
        self.p_zero = None         # posición (x,y,z) de pose neutral
        self.pressure_zero = 0.0   # offset de presión en reposo
        self.flow_max_user = 1.0   # flujo máximo del usuario (calibrado)

        # bisel virtual en el espacio del usuario (frame neutral)
```

```
        # por defecto: 20 cm al frente del origen neutral, mismo alto
        self.P_bisel = np.array([0.0, 0.0, -self.bisel_offset_m])

        # buffers
        self.p_buf = deque(maxlen=512)  # presión para FFT de vibrato

    def run(self):
        ser = serial.Serial(self.port, 921600, timeout=0.05)
        cam_thread = self._start_camera_thread()

        t_next = time.perf_counter()
        while self.running:
            pkt = self._read_packet(ser)
            if pkt is None:
                continue
            self._update_sensor_state(pkt)

            now = time.perf_counter()
            if now >= t_next:
                self._publish_to_musician()
                t_next = now + 0.005    # 200 Hz

    def _update_sensor_state(self, pkt):
        # quaternion de orientación de la cabeza
        self.q_head = np.array([pkt.qw, pkt.qx, pkt.qy, pkt.qz])
        # presión en Pa (convertida desde ADC)
        self.pressure_pa = self._adc_to_pa(pkt.pressure_adc) - self.pressure_zero
        self.p_buf.append((pkt.t_us, self.pressure_pa))
        # botones
        self.touch_state = pkt.touch

    def _publish_to_musician(self):
        # posición de la cabeza (desde cámara + ArUco)
        P_head = self._aruco_xyz()  # 3D absoluto en frame del atril

        # aplicar mapeo magnificado
        l, theta, offset = magnified_flute_mapping(
            P_head, self.q_head, self.P_bisel,
            q_zero=self.q_zero, p_zero=self.p_zero, k=self.k)

        # flujo y vibrato desde la señal de presión
        flow = self._pressure_to_flow(self.pressure_pa)
        vib_f, vib_a = self._vibrato_from_buffer()

        self.musician_pipe.send({
            "type": "helmet_ref",
            "l": l, "theta": theta, "offset": offset,
            "flow": flow,
            "vib_freq": vib_f, "vib_amp": vib_a,
```

```
            "touch": self.touch_state,
        })

    def calibrate_zero(self):
        """Llamado desde GUI. Promedia 1 s de datos como zero-pose."""
        ...
```

8.2 Función de mapeo magnificado

```python
def magnified_flute_mapping(P_head, q_head, P_bisel, q_zero, p_zero, k=10.0):
    """
    Mapea pose de la cabeza (P_head, q_head) a coordenadas del embocador
    del robot (l, theta, offset), dado un bisel virtual P_bisel y un factor
    de magnificación k.

    Frame: X lateral (positivo a la derecha del usuario), Y vertical,
    Z hacia adelante (negativo = frente al usuario).
    """
    # 1. pose relativa a zero-pose
    q_rel = quat_mul(quat_inv(q_zero), q_head)
    P_rel = P_head - p_zero
    R_head = R.from_quat([q_rel[1], q_rel[2], q_rel[3], q_rel[0]])

    # 2. vector del chorro desde la boca
    MOUTH_OFFSET = np.array([0.0, -0.05, -0.08])  # boca respecto al vertex
    P_mouth = P_rel + R_head.apply(MOUTH_OFFSET)
    v_jet = R_head.apply(np.array([0.0, 0.0, -1.0]))

    # 3. geometría respecto al bisel virtual
    to_bisel = P_bisel - P_mouth
    dist_mouth_bisel = np.linalg.norm(to_bisel)

    # 4. punto del ray del chorro más cercano al bisel
    s_closest = np.dot(to_bisel, v_jet)  # parámetro a lo largo del ray
    P_closest = P_mouth + s_closest * v_jet
    miss_vector = P_closest - P_bisel    # cuánto "falla" el ray al bisel

    # 5. mapeo magnificado a parámetros del robot
    l_robot = dist_mouth_bisel / k
    theta_robot = np.arctan2(miss_vector[1], s_closest)  # componente vertical
    offset_robot = miss_vector[0] / k                    # componente lateral

    return l_robot, theta_robot, offset_robot
```

8.3 Cambios en `main.py`

```python
# main.py — añadir al arranque
from src.helmet import HelmetDriver
```

```
helmet_musician_pipe, musician_helmet_pipe = Pipe()
helmet = HelmetDriver(
    port="COM7",
    musician_pipe=helmet_musician_pipe,
    cam_index=0,
    bisel_offset_m=0.20,
    magnification_k=10.0,
)
helmet.start()

musician = Musician(
    ...,
    helmet_pipe=musician_helmet_pipe,
)
```

8.4 Cambios en `src/musician.py`

```
# dentro del loop principal del musician
if self.mode == "helmet_live":
    while self.helmet_pipe.poll():
        msg = self.helmet_pipe.recv()
        if msg["type"] == "helmet_ref":
            # convertir (l, theta, offset) a (x, z, alpha) vía cinematica
            x, z, alpha = self.cinematica.task_to_joint(
                msg["l"], msg["theta"], msg["offset"])
            self.x_driver.set_reference(x)
            self.z_driver.set_reference(z)
            self.alpha_driver.set_reference(alpha)
            self.flow_driver.set_reference(msg["flow"])
            self.vibrato_driver.set_reference(
                msg["vib_freq"], msg["vib_amp"])
```

8.5 Cambios en la GUI

- Nuevo botón **“Helmet Mode”** junto al PS Move existente.
- Widget de calibración zero-pose: el usuario mira al frente en postura neutral, pulsa “Zero”, y se graban los offsets.
- Slider para ajustar el **factor de magnificación k** (3 - 15) en tiempo real, para que cada flautista encuentre su rango cómodo.
- Visor live: dibuja un esquema 2D vista superior y vista lateral con la cabeza del usuario y el bisel virtual, mostrando en tiempo real el ray del chorro y dónde impacta (o falla) el bisel.
- Indicador de estado: LEDs verde (ok), amarillo (ArUco perdido), rojo (USB caído).

9. Calibración y mapeo de gestos

9.1 Calibración de instalación (una sola vez)

Paso	Qué se calibra	Duración
1	Geometría de cámara (matriz K intrínseca, OpenCV calibrateCamera, patrón ajedrez)	30 min
2	Pose de cámara respecto al espacio del robot (ArUco grande fijado al robot)	5 min
3	Orientación del IMU respecto al cuerpo del casco (hand-eye calibration)	15 min
4	Transferencia del sensor de presión (Pa vs ADC) con manómetro referencia	20 min
5	Posición neutral del atril respecto a cámara y definición de `P_bisel`	10 min

Total: ~1.5 h de calibración inicial.

9.2 Calibración por sesión (cada vez que se enciende)

- **Zero-pose**: usuario se pone el casco, se sienta en posición cómoda frente al atril, mira al frente, pulsa "Zero". Se promedian 1 s de orientación y posición.
- **Rango de flujo personal**: se pide "sopla suave" (3 s), "sopla fuerte" (3 s). Se define mapeo afín $P_{\text{usuario}} \rightarrow$ flujo robot.
- **Ajuste del factor** k: el usuario mueve la cabeza mientras ve en la GUI el movimiento del robot; ajusta el slider hasta sentirse cómodo.
- **Chequeo de tracking ArUco**: usuario mueve la cabeza a 5 posiciones marcadas; se valida error < 5 mm.

Total: ~60 segundos de calibración por sesión.

9.3 Construcción de diccionario de notas (caso de uso clave)

El flujo actual con PS Move es: mover control -> escuchar -> ajustar -> guardar con botón. Con el casco es equivalente pero mucho más natural:

```
1. Usuario: se pone el casco, calibra zero-pose, coloca los dedos
   (en control manual o MIDI) y orienta la cabeza hacia el bisel
   virtual.
2. Usuario: sopla como lo haría para la nota objetivo.
3. Robot: replica en tiempo real los parámetros mapeados
   (l, theta, offset, flow).
4. Microfono del lab: captura el sonido del robot.
5. Usuario: ajusta sutilmente girando/inclinando la cabeza hasta
   que la nota suena afinada y timbrada.
6. Usuario: presiona el botón capacitivo en la sien
```

```
   ("grabar nota") — sin soltar nada, sin bajar los brazos.
7. Sistema: guarda {nota, x, z, alpha, flow, vib, dedos} en el
   diccionario.
8. Próxima nota.
```

Bonus: el botón está en la sien, así que el usuario no tiene que mover las manos ni interrumpir el gesto de soplo para guardar. Mejora de usabilidad sustancial respecto al PS Move.

10. Evaluación honesta de capacidades

> **Aviso.** Esta sección es deliberadamente pesimista. Separamos lo que estamos bastante seguros de capturar, lo dudoso, y lo que probablemente no capturaremos.

10.1 Zona VERDE — medibles con alta confianza

Variable	Fuente	Exactitud esperada
Orientación de cabeza (yaw/pitch/roll)	BNO085 fusion	< 1° RMS, drift < 2°/h
Posición 3D (x, y, z)	ArUco múltiple + cámara	< 2 mm con 3 marcadores 40 mm a 1 m
Presión aire media (DC)	MPX5010DP + filtro 10 Hz	±5 Pa en rango 0-3 kPa
Botones capacitivos	ESP32 touch	Robusto, sin rebote

Estos nos permiten mapear los primeros 5 grados de libertad del robot (x, z, α, flujo, y usuario via botón) con confianza.

10.2 Zona AMARILLA — depende de detalles de implementación

Variable	Riesgo
Vibrato en el aliento	El pitot atenúa la componente AC. Frecuencias 5-7 Hz típicas de vibrato flautístico deberían pasar, pero a SNR variable. Probabilidad de éxito ~70 % (mejor que los 55 % de la propuesta anterior, porque la geometría es fija).
Ataques de lengua	Transitorios < 20 ms. El pitot tiene latencia y suavizado por el tubo. Detectaremos ataques fuertes pero no las sutilezas (single, double, flutter). Probabilidad ~40 %.
Vibrato bi-componente (AM + FM)	Algunos flautistas modulan también ligeramente el ángulo del jet, lo cual sí capturamos vía IMU en el pitch. Requiere desarrollo.

Variable	Riesgo
Coordinación cabeza-mano-dedos	El casco no tiene dedos; si queremos tocar con dedos reales simultáneamente, el músico debe operar otro dispositivo. Viable pero es alcance fuera de v1.

10.3 Zona ROJA — NO medibles con esta propuesta

Variable	Por qué no
Forma de apertura labial	Depende de geometría interna de labios/boca del usuario. Mediríamos con cámara infrarroja apuntando a los labios, o EMG labial (fuera de alcance).
Focalización del chorro (jet width)	Un flautista controla si el chorro es “ancho y disperso” o “angosto y concentrado”. Esto NO tiene correlato directo en orientación de la cabeza.
Turbulencia / ruido del jet	La textura del sonido depende de micro-perturbaciones del aire. Imposible captar en el pitot lateral.
Resonancia de la cavidad bucal	Algunos flautistas usan la cavidad bucal como filtro (vowel morphing). No transferible.

10.4 Conclusión honesta

> Lo que obtenemos es «intuición de flautista para los controles gruesos y medios». El usuario usa su cuerpo como flautista, sopla como flautista, y el robot responde con razonable fidelidad en los 5-6 grados de libertad que dominan el 80 % de la expresividad. Para el 20 % restante (matiz tímbrico fino) seguiremos necesitando el oído del músico como feedback loop — exactamente como hoy.

Respecto al PS Move, esta es una mejora enorme en intuición, robustez del stack, y calidad de captura del soplo. Respecto al cabezal sensorizado descartado, es más limpia arquitectónicamente y más ergonómica.

11. Plan de validación fail-fast

Antes de invertir 6 semanas en construir el sistema completo, validamos los supuestos más riesgosos con experimentos pequeños.

Test H.1 — ¿El IMU en la cabeza es suficientemente estable? (2 días)

- Setup: BNO085 sobre protoboard, cinta adhesiva a un casco de bicicleta como placeholder, ESP32, streaming por USB.

- Procedimiento: usuario se sienta, mira al frente, no se mueve durante 30 min. Se graba yaw/-pitch/roll. Se repite con movimientos de cabeza naturales (no tocar flauta aún).
- Criterio de éxito: drift < 2° en 30 min; RMS de ruido en reposo < 0.2°.
- Si falla: habilitar magnetómetro del BNO085 (modo rotation_vector completo en vez de game_rotation_vector). Recalibrar periódicamente.

Test H.2 — ¿El pitot montado en boom detecta vibrato? (3 días)

- Setup: pitot fijado a un arnés improvisado (cinta, cartón) a 4 cm de los labios; sensor MPX5010DP conectado a ESP32.
- Procedimiento: flautista sopla 30 s sin vibrato y 30 s con vibrato clásico (5-7 Hz, amplitud modesta). FFT de la señal.
- Criterio de éxito: pico claro a 5-7 Hz con SNR > 10 dB.
- Si falla: upgrade a SDP810 (+ US$60) o cambiar topología del pitot (malla de captura en vez de lateral).

Test H.3 — ¿ArUco múltiple sobrevive a rotaciones de cabeza? (2 días)

- Setup: 3 marcadores ArUco impresos 40 mm, fijados al casco placeholder en frontal, sien izquierda y sien derecha. Cámara C920 a 1 m.
- Procedimiento: usuario gira cabeza ±90°, inclina ±45°, se traslada ±20 cm. Registrar continuidad del tracking.
- Criterio de éxito: al menos 1 marcador detectado en todo momento; pose reconstruida con jitter < 2 mm en posición estática.
- Si falla: subir a marcadores de 50 mm; agregar 4° marcador en occipital; mejor iluminación.

Test H.4 — Prueba conceptual offline (3 días) — EL MÁS IMPORTANTE

- Setup: IMU simple (celular con app `SensorLogger` o `Gyr0SC`), receptor Python, visualizador 3D mostrando bisel virtual y jet.
- Procedimiento: el flautista (idealmente Patricio) se pone el celular en la cabeza con una vincha elástica, mueve la cabeza como tocaría, visualiza en pantalla el bisel virtual y el ray del chorro. Opcionalmente, un modelo físico de flauta (Flute Physical Model en Max/MSP o en Python) sonifica los parámetros mapeados.
- Criterio de éxito: el flautista reporta “se siente natural apuntar al bisel con la cabeza”; puede completar ejercicios simples de cambio de octava y dinámica sin frustración.
- Si falla: **reconsiderar toda la propuesta antes de construir hardware.** Es el test más barato y el que más información aporta.

Test H.5 — Latencia end-to-end (1 día)

- Setup: prototipo básico + LED en ESP32 + cámara lenta.
- Procedimiento: generar un impulso brusco en el IMU, registrar con LED el momento, filmar respuesta del motor.
- Criterio: < 30 ms desde gesto hasta inicio del movimiento del robot.

Test H.6 — Aceptación y confort (2 días)

- Setup: prototipo funcional + diccionario de 10 notas por construir.

- Procedimiento: un flautista externo (no Patricio) usa el casco durante 30 min. Registra comentarios subjetivos de confort y facilidad.
- Criterio: completa 10 notas sin fatiga visible ni quejas ergonómicas; comenta que “se siente intuitivo”.
- Si falla: rediseñar arnés, mapeos, o factor de magnificación.

Gate check. H.4 es el test más importante y se puede ejecutar AHORA con hardware mínimo (celular + laptop). Si H.4 no convence, todo lo demás se suspende.

12. Fases de implementación

Fase	Duración	Entregable	Dependencias
H.0 Prueba conceptual	3 días	Video + reporte de H.4 con un flautista convencido	Hardware: celular + laptop
H.1 Validación sensores	1 sem	Tests H.1-H.3 pasando en paralelo	H.0 ok
H.2 Firmware ESP32	1 sem	Trama a 250 Hz por USB con CRC	H.1 ok
H.3 Driver Python	1 sem	HelmetDriver estable + fusión con ArUco	H.2 ok
H.4 Integración GUI/musician	1 sem	Modo “Helmet Live” en GUI funcional	H.3 ok
H.5 Diseño definitivo del arnés	1 sem	Arnés impreso + ensamblaje final + trampa agua	En paralelo con H.3-H.4
H.6 Calibración + diccionario	1 sem	Diccionario de 10 notas construido con el casco	H.4-H.5 ok
H.7 Hardening	1 sem	Tests de regresión + manual de usuario	H.6 ok

Total estimado: **~7 semanas-persona** si todo sale bien.

13. Estimaciones de éxito y riesgos

13.1 Probabilidades subjetivas

Objetivo	Probabilidad	Racional
H.0 (concepto) convence a Patricio	75 %	La metáfora es sólida, pero siempre hay sorpresas al probar
Hardware funcional y estable	85 %	Partes estándar, bien documentadas

Objetivo	Probabilidad	Racional
Mapeos básicos (orient, pos, flow) funcionan bien	90 %	Física directa, sin ambigüedades
Vibrato detectado con calidad útil	70 %	Mejor que 55 % del cabezal — geometría fija ayuda
Experiencia “se siente como una flauta”	80 %	Apuesta con mayor incertidumbre; depende del ajuste de k y del músico
Construcción eficiente de diccionarios	85 %	Caso de uso clave; el botón en sien es un win claro
Sistema estable en demos públicas	75 %	Depende del hardening (fase H.7)
Confort en sesiones de 30+ min	70 %	250 g es aceptable pero no trivial

13.2 Riesgos técnicos principales

- **Drift del yaw sin magnetómetro.** El BNO085 en modo game-rotation-vector no usa mag; puede derivar 1-2°/h. Mitigación: recalibrar zero-pose cada pocos minutos, o habilitar mag con calibración inicial.
- **Oclusión de ArUcos.** Si el usuario baja mucho la cabeza puede tapar los marcadores con sus hombros. Mitigación: 3+ marcadores distribuidos; fallback a IMU integrado.
- **Humedad del aliento en el pitot.** Problema real con flautistas — aliento húmedo condensa. Mitigación: trampa de agua en el tubo, limpieza periódica (cada 20 min).
- **Fatiga cervical.** Sostener la cabeza ligeramente adelantada durante sesiones largas cansa. Mitigación: apoyacabezas parcial o silla ergonómica; pausas programadas.
- **Acoplamiento respiración-pose.** Al respirar, la cabeza se mueve. Mitigación: filtro pasa-bajos de 2 Hz en posición (el usuario no apunta a 2 Hz de frecuencia).
- **Ruido EMI del cable USB en el ADC de presión.** Mitigación: cable apantallado, filtro RC en la entrada del ADC, tierra común bien diseñada.

13.3 Riesgos de proyecto

- **Scope creep en el diseño del casco.** La tentación de agregar “otro sensor más” es fuerte. Mitigación: H.0 congela el scope hasta pasar.
- **Dependencia de un solo flautista para H.6.** Si la intuición varía entre flautistas, hay sesgo. Mitigación: validar con 2-3 flautistas distintos antes de fase final.
- **Costo en horas de diseño mecánico.** Un buen arnés requiere iteración. Presupuestar 2-3 versiones impresas antes de la final.

14. Comparación con alternativas evaluadas

Criterio	PS Move actual	Cabezal sensorizado (v0.1)	**Casco magnificado**
Intuición musical	Baja	Alta (es una flauta)	**Muy alta (es el cuerpo del flautista)**
Calidad sensor soplo	N/A (no mide soplo)	Media (pitot en objeto móvil)	**Alta (geometría fija)**
Tracking 3D	Color HSV (frágil)	ArUco en cabezal chico	**ArUco múltiple en casco rígido**
Manos libres	No	No	**Sí**
Permite tocar dedos reales en el futuro	No	No	**Sí**
Ergonomía sostenida	Media	Media (sostener peso en mano)	**Alta (peso en cabeza bien distribuido)**
Hardware discontinuado	Sí	No	No
Costo hardware	0 (tenemos PS Move)	US$ 90-280	US$ 120-200
Tiempo de desarrollo	0	~6 sem	~7 sem
Valor demo pública	Medio	Alto	**Muy alto** (visualmente impactante, musicalmente legible)
Publicable (NIME / ICMC / NIME)	No	Sí	**Sí, novedoso: "embodied magnified flute interface"**

15. Decisiones abiertas

Estas son decisiones que dependen del contexto del laboratorio y preferencias del equipo. Se listan para discutir antes de empezar.

D.1 — ¿Cableado USB o wireless?

- **USB CDC (cableado)**: latencia predecible, sin problemas de radio. Cable fino tipo micro-USB a USB-C sale por la nuca. Viable.
- **BLE / Wi-Fi**: experiencia mucho más fluida; requiere batería LiPo + cargador. Latencia variable.
- **Recomendación**: USB para H.0-H.4 (debuggeable). BLE como upgrade opcional en H.7 si se justifica para demos.

D.2 — ¿Cámara en el atril o en el techo?

- **Atril**: más cerca, mejor resolución de ArUcos, portátil.
- **Techo**: vista superior, menos oclusión, pero más fija.
- **Recomendación**: atril para v1. Techo como estudio futuro.

D.3 — ¿Mantener el modo PS Move o reemplazarlo?

- Mantener ambos durante 6 meses después de H.7: backup útil, permite comparar experiencias.

- Luego deprecar PS Move si el casco demuestra valor.

D.4 — ¿Quién ejecuta cada fase?

- Firmware / electrónica: ideal un tesista o ayudante con formación en embebidos.
- Driver / integración Python: Claude Code + revisión de Patricio.
- Diseño mecánico del arnés: idealmente un estudiante de diseño industrial o ingeniería mecánica.
- Calibración / mapeos musicales: Patricio.
- Validación H.6: 2-3 flautistas externos.

D.5 — ¿Publicación?

Material suficiente para un paper en **NIME (New Interfaces for Musical Expression)** o **ICMC**. Ángulos interesantes:

- "Magnified embodied interface for a robotic flute player"
- Comparación cuantitativa PS Move vs casco en construcción de diccionarios.
- Resultados de aceptación con múltiples flautistas (H.6).

D.6 — Presupuesto

Hardware en el rango **US$ 120-200**. Tiempo-persona es el costo dominante: ~7 sem × 1 persona = 280 h. Con Claude Code como par, el tiempo-persona efectivo baja a ~180 h.

D.7 — ¿Casco abierto (arnés) o cerrado (casco completo)?

- **Arnés abierto (recomendado)**: ligero, ventilado, no tapa orejas.
- **Casco cerrado tipo football**: más estable pero pesado (>1 kg), caluroso, impide escuchar al robot con fidelidad.
- **Recomendación firme**: arnés abierto.

Próximos pasos sugeridos

1. **AHORA**: Ejecutar H.0 — prueba conceptual offline con celular + laptop. ~3 días.
2. Decisión sobre D.1-D.3 y D.7 con colegas.
3. Si H.0 convence: comprar kit mínimo (~US$ 60) y ejecutar H.1-H.3 en paralelo (1 semana).
4. Go/no-go basado en resultados de H.0 y H.1-H.3 antes de seguir con H.2-H.7.

Cierre. El valor principal de esta propuesta no es solo mejorar la ergonomía del controlador, sino alinear la interfaz con la intuición corporeizada del músico. Pierre Flutist deja de ser «un robot que se opera» y se convierte en «un robot que el flautista toca con su cuerpo» — un cambio filosófico que abre puertas para investigación en performance, pedagogía y co-creación humano-máquina.